

Gymnázium, Praha 6, Arabská 14
Obor programování



Ročníkový projekt

Šachy

Nazarii Pitsur, 1.E

Mgr. Jan Lána

Duben 2026

Prohlašuji, že jsem jediným autorem tohoto projektu, všechny citace jsou řádně označené a všechna použitá literatura a další zdroje jsou v práci uvedené. Tímto dle zákona 121/2000Sb. (tzv. Autorský zákon) ve znění pozdějších předpisů uděluji bezúplatně škole Gymnázium, Praha 6, Arabská 14 oprávnění k výkonu práva na rozmnožování díla (§ 13) a práva na sdělování díla veřejnosti (§ 18) na dobu časově neomezenou a bez omezení územního rozsahu.

V dne

Nazarii Pitsur.....

Anotace: Projekt se zabývá vytvořením šachové hry v jazyce Java s využitím JavaFX. Byla implementována herní logika včetně pohybu figurek, střídání hráčů a základních pravidel šachu, jako jsou rošáda, en passant a proměna pěšce. Program také rozpoznává šach, mat a pat. Grafické rozhraní umožňuje ovládání hry pomocí klikání na šachovnici a zobrazuje aktuální stav hry. Výsledkem je funkční aplikace pro hraní šachu.

Abstract: The project focuses on creating a chess game in the Java programming language using JavaFX. The game logic includes piece movement, turn switching, and basic chess rules such as castling, en passant, and pawn promotion. The program also detects check, checkmate, and stalemate. The graphical interface allows the user to control the game by clicking on the chessboard and displays the current game state. The result is a functional application for playing chess.

Obsah

1 Úvod.....	1
2 Použité technologie.....	1
3 Datové struktury a třídy.....	1
3.1 Figurka (Třída Figurka).....	1
3.2 Šachovnice (Třída Sachovnice).....	2
3.3 Pozice (Třída Pozice).....	2
3.4 JavaFX zobrazení (Třída JavaFxtirda).....	2
3.5 Textury (Resources složka textury).....	2
3.6 Jednotlivé figurky.....	2
3.6.1 Pěšec(Třída Pesec).....	2
3.6.2 Král(Třída Kral).....	2
3.6.3 Kůň(Třída Kun).....	2
3.6.4 Věž(Třída Vez).....	3
3.6.5 Střelec(Třída Strelec).....	3
3.6.6 Královna(Třída Dama).....	3
4 Klíčové algoritmy.....	3
4.1 Generování možných tahů.....	3
4.2 Kontrola šachu.....	3
4.3 Validace tahu.....	3
4.4 Mat.....	3
4.5 Pat.....	3
4.6 Rošáda.....	4
4.7 En passant.....	4
4.8 Proměna pěšce.....	4
5 Herní logika.....	4
5.1 Střídání hráčů.....	4
5.2 Výběr a provedení tahu.....	4
5.3 Kontrola platnosti tahu.....	4
5.4 Aktualizace stavu hry.....	4
5.5 Kontrola konce hry.....	5
5.6 Speciální tahy.....	5
6 Uživatelské rozhraní.....	5
6.1 Šachovnice.....	5
6.2 Zobrazení figurek.....	5
6.3 Ovládání.....	5
6.4 Zvýraznění.....	5
6.5 Aktualizace.....	5
6.6 Informace pro hráče.....	5
6.7 Tlačítka.....	5
7 Problémy a řešení.....	6

7.1 Validace tahů.....	6
7.2 Kontrola šachu.....	6
7.3 Implementace pohybu figurek.....	6
7.4 Aktualizace šachovnice.....	6
7.5 Ovládání pomocí klikání.....	6
7.6 Speciální tahy.....	6
8 Možnosti rozšíření.....	6
9 Závěr.....	7
10 Zdroje.....	7

1 Úvod

Tato práce se zabývá vytvořením šachové hry v programovacím jazyce Java s využitím knihovny JavaFX. Cílem projektu bylo vytvořit funkční aplikaci, která umožňuje hru dvou hráčů podle základních pravidel šachu.

Důraz byl kladen na implementaci herní logiky, zejména pohybu figurek, validace tahů a detekce herních situací, jako je šach, mat a pat. Součástí programu je také grafické uživatelské rozhraní, které umožňuje ovládání hry pomocí myši.

V následujících kapitolách jsou popsány použité technologie, třídy a algoritmy.

Zadání:

Téma: Šachy

Popis:

Pole 8x8, základní pravidla pohybu, změna pěšce na královnu po dosažení druhé strany pole, vyhrává ten kdo první zabije krále. Hru budou hrát 2 hráči pomocí myši, strany se budou střídát (bílá a černá). Hráč klikne myší na postavu a pak klikne na pole kam ji chce přemístit.

2 Použité technologie

Program byl vytvořen v programovacím jazyce Java. K vývoji byla použita IntelliJ IDEA.

Grafické uživatelské rozhraní je realizováno pomocí knihovny JavaFX. Pro zobrazení šachovnice byla použita komponenta GridPane, jednotlivá pole jsou reprezentována tlačítky (Button) a figurky jsou zobrazovány pomocí komponenty ImageView.

Program využívá standardní knihovny jazyka Java. Pro ukládání možných tahů jednotlivých figurek je použita kolekce typu List.

3 Datové struktury a třídy

3.1 Figurka (Třída Figurka)

Uchovává, jestli je figurka bílá nebo černá, a určuje, že každá figurka musí mít vlastní metodu pro výpočet možných tahů.

3.2 Šachovnice (Třída Sachovnice)

Třída představuje hlavní herní část programu. Obsahuje dvourozměrné pole `Figurka[][]`, které reprezentuje šachovnici. Zajišťuje provádění tahů, kontrolu jejich platnosti a vyhodnocení herních situací, jako je šach, mat a pat.

3.3 Pozice (Třída Pozice)

Třída `Pozice` slouží k reprezentaci souřadnic na šachovnici. Obsahuje hodnoty `x` a `y`, které určují konkrétní pole.

3.4 JavaFX zobrazení (Třída JavaFxtirda)

Třída zajišťuje zobrazení šachovnice a komunikaci s hráčem. Vytváří grafické rozhraní, ve kterém je šachovnice zobrazena jako mřížka tlačítek.

Na jednotlivých polích jsou zobrazeny obrázky figurek. Hráč ovládá hru pomocí klikání – nejprve vybere figurku a poté cílové pole. Program následně provede tah a aktualizuje stav hry.

Třída také zvýrazňuje vybranou figurku a její možné tahy a zobrazuje, který hráč je na tahu. Po každém tahu se šachovnice znovu vykreslí, aby odpovídala aktuálnímu stavu hry.

3.5 Texturey (Resources složka texturey)

Složka obsahuje grafické soubory jednotlivých šachových figurek. Tyto obrázky jsou načítány během běhu programu a zobrazovány na šachovnici pomocí komponenty `ImageView`.

3.6 Jednotlivé figurky

3.6.1 Pěšec(Třída Pesec)

Pěšec se pohybuje o jedno pole dopředu, z výchozí pozice může o dvě pole. Bere diagonálně. Obsahuje také speciální pravidla, jako je *en passant* a proměna na jinou figurku po dosažení poslední řady.

3.6.2 Král(Třída Kral)

Král se pohybuje o jedno pole ve všech směrech. Obsahuje také logiku pro rošádu a informaci o tom, zda se již pohnul.

3.6.3 Kůň(Třída Kun)

Kůň se pohybuje ve tvaru písmene „L“ (dvě pole jedním směrem a jedno kolmo). Jako jediná figurka může přeskakovat ostatní figurky.

3.6.4 Věž(Třída Vez)

Věž se pohybuje vodorovně a svisle o libovolný počet polí, dokud nenarazí na jinou figurku. Obsahuje informaci o tom, zda se již pohnula, což je důležité pro rošádu.

3.6.5 Střelec(Třída Strelec)

Střelec se pohybuje diagonálně o libovolný počet polí, dokud nenarazí na jinou figurku.

3.6.6 Královna(Třída Dama)

Královna kombinuje pohyb věže a střelce. Může se pohybovat vodorovně, svisle i diagonálně o libovolný počet polí.

4 Klíčové algoritmy

4.1 Generování možných tahů

Každá figurka obsahuje metodu, která vrátí seznam možných tahů podle pravidel šachu. Algoritmus kontroluje hranice šachovnice a přítomnost ostatních figurek. U lineárních figurek (věž, střelec, dáma) se prochází pole v daném směru, dokud není pohyb blokován.

4.2 Kontrola šachu

Program nejprve vyhledá pozici krále. Následně projde všechny nepřátelské figurky a získá jejich možné tahy. Pokud některý tah odpovídá pozici krále, je král v šachu.

4.3 Validace tahu

Při pokusu o tah je tah nejprve simulován. Program provede tah nanečisto a následně zkontroluje, zda hráč nezůstane v šachu. Pokud ano, tah je označen jako neplatný a vrátí se zpět.

4.4 Mat

Mat nastává v případě, že je král v šachu a zároveň neexistuje žádný platný tah, kterým by se hráč mohl bránit. Program proto kontroluje všechny možné tahy daného hráče a ověřuje, zda některý z nich nevede k bezpečné pozici.

4.5 Pat

Pat nastává, pokud hráč není v šachu, ale zároveň nemá žádný platný tah. Program tuto situaci vyhodnocuje podobně jako mat, ale bez přítomnosti šachu.

4.6 Rošáda

Rošáda je povolena pouze v případě, že se král ani příslušná věž ještě nepohnuli a mezi nimi nejsou žádné figurky. Dále nesmí být král v šachu ani procházet přes ohrožená pole. Pokud jsou všechny podmínky splněny, dojde k přesunu krále a věže.

4.7 En passant

En passant je speciální tah pěšce, který je možný bezprostředně po tahu soupeřova pěšce o dvě pole. Pokud se pěšec nachází vedle něj, může ho vzít, jako by se posunul pouze o jedno pole.

4.8 Proměna pěšce

Proměna pěšce nastává ve chvíli, kdy pěšec dosáhne poslední řady šachovnice. V takovém případě je nahrazen jinou figurkou, královnou.

5 Herní logika

5.1 Střídání hráčů

Hra je založena na střídání dvou hráčů. Aktuální hráč je určen proměnnou, která udává, zda je na tahu bílý nebo černý hráč. Po každém platném tahu se tato hodnota změní.

5.2 Výběr a provedení tahu

Hráč provádí tah pomocí klikání na šachovnici. Nejprve vybere figurku a poté cílové pole. Program následně zavolá metodu pro provedení tahu, která rozhodne o jeho platnosti.

5.3 Kontrola platnosti tahu

Program nejprve ověří, zda figurka patří aktuálnímu hráči. Poté získá všechny možné tahy dané figurky. Pokud zvolený tah odpovídá některému z nich, je tah dále ověřen simulací, aby nedošlo k vlastnímu šachu.

5.4 Aktualizace stavu hry

Po provedení platného tahu dojde k přesunu figurky na cílové pole a případnému odebrání soupeřovy figurky. V případě pěšce může dojít k proměně na jinou figurku. Následně se aktualizuje stav šachovnice.

5.5 Kontrola konce hry

Po každém tahu program kontroluje, zda nastala některá z herních situací. Pokud je hráč v šachu a nemá žádný platný tah, jedná se o mat. Pokud hráč není v šachu, ale nemá žádný tah, jedná se o pat.

5.6 Speciální tahy

Součástí herní logiky jsou také speciální tahy, jako je rošáda, en passant a proměna pěšce, které jsou zpracovány při provádění tahu podle stanovených podmínek.

6 Uživatelské rozhraní

6.1 Šachovnice

Šachovnice je zobrazena jako mřížka 8×8 pomocí komponenty GridPane. Jednotlivá pole jsou reprezentována tlačítky a mají střídavé barvy.

6.2 Zobrazení figurek

Figurky jsou zobrazeny pomocí obrázků, které jsou vkládány do tlačítek pomocí komponenty ImageView. Každá figurka má vlastní obrázek podle typu a barvy.

6.3 Ovládání

Ovládání hry probíhá pomocí myši. Hráč nejprve klikne na figurku a poté na cílové pole. Program následně vyhodnotí a provede tah.

6.4 Zvýraznění

Vybraná figurka je zvýrazněna odlišnou barvou. Možné tahy jsou zvýrazněny, aby měl hráč přehled o dostupných pohybech.

6.5 Aktualizace

Po každém tahu se šachovnice znovu vykreslí, aby odpovídala aktuálnímu stavu hry.

6.6 Informace pro hráče

Rozhraní zobrazuje, který hráč je na tahu, a informuje o konci hry v případě matu nebo patu.

6.7 Tlačítka

Součástí rozhraní jsou ovládací tlačítka, například pro restart hry, nabídku remízy a ukončení programu.

7 Problémy a řešení

7.1 Validace tahů

Bylo nutné zajistit, aby hráč nemohl provést neplatný tah, například takový, který by vedl k vlastnímu šachu. Řešením bylo simulování tahu nanečisto a následná kontrola pomocí metody pro zjištění šachu.

7.2 Kontrola šachu

Program musí správně určit, zda je král ohrožen. Tento problém byl řešen vyhledáním pozice krále a následným procházením všech nepřátelských figurek a jejich možných tahů.

7.3 Implementace pohybu figurek

Každá figurka má odlišná pravidla pohybu. Řešením bylo vytvoření společné třídy Figurka a využití dědičnosti, kde každá figurka implementuje vlastní logiku pohybu.

7.4 Aktualizace šachovnice

Po provedení tahu bylo nutné správně zobrazit aktuální stav hry. Řešením bylo opětovné vykreslení celé šachovnice po každém tahu.

7.5 Ovládání pomocí klikání

Bylo potřeba zajistit intuitivní ovládání hry. Řešením bylo použití dvou kliknutí – první pro výběr figurky a druhý pro určení cílového pole.

7.6 Speciální tahy

Implementace speciálních tahů byla složitější než běžné tahy. Řešením bylo přidání podmínek, které kontrolují, zda jsou splněny všechny požadavky pro jejich provedení.

8 Možnosti rozšíření

Program by bylo možné dále rozšířit o další funkce. Mezi možné rozšíření patří přidání umělé inteligence pro hru proti počítači, ukládání a načítání rozehraných her nebo implementace online multiplayeru. Dále by bylo možné vylepšit grafické uživatelské rozhraní a přidat pokročilejší vizuální prvky.

9 Závěr

Cílem projektu bylo vytvořit funkční šachovou hru v programovacím jazyce Java s grafickým uživatelským rozhraním. Tento cíl se podařilo splnit. Program umožňuje hru dvou hráčů podle základních pravidel šachu a správně vyhodnocuje herní situace, jako je šach, mat a pat.

Během práce byly řešeny problémy spojené s implementací herní logiky a uživatelského rozhraní. Projekt přispěl k lepšímu pochopení programování a práce s grafickými aplikacemi.

10 Zdroje

[1] Textury postav: [https://cs.wikipedia.org/wiki/Jezdec_\(%C5%A1achy\)](https://cs.wikipedia.org/wiki/Jezdec_(%C5%A1achy))

[2] Logo školy: <https://github.com/gyarab>

[3] ChatGPT prompt: <cau ted delam projekt v java a jsou to sachy, uz jsem vytvoril tridu figurrka ktera ma boolean je bila a vytvoril jsem tridy s pescem konem a ostatnima typama figurek ktere v sachach jsou, udelal jsem tridu sachovnice a tam pomoci policko[][] a jeBila true/false vyskladal figurky do policek na kterych zacinaji a nevim co mam delat ted, ty sachy chci udelat tak aby pak sli dat to java fx > datum: přibližně 16 dubna 2026 (<https://chatgpt.com>)

[4] Konzultace problémů pomocí jazykových modelů ChatGPT a Gemini. Použito pro vysvětlení a návrh řešení.

Veškerý kód byl vytvořen a upraven autorem práce.